Multi-Source Candidate Data Transformer

Backend Engineering Design Document | Muskan Sarraf | muskansarraf328@gmail.com

Problem Framing

Eightfold receives candidate data from messy, overlapping sources. The transformer produces one canonical candidate profile per person with normalized values, deterministic merge behavior, provenance for every extracted field, and an honest confidence score. The project prioritizes correctness and explainability over UI complexity.

1. Extract	2. Normalize	3. Deduplicate	4. Merge	5. Confidence	6. Project	7. Validate
ATS JSON, recruiter CSV, GitHub snapshot, recruiter notes	Email, phone, country, date, skills	Email first, phone second, conservative name fallback	Pick trusted scalars, union list fields	Score by source and field coverage	Runtime config remaps output	Required fields and types

Canonical Schema And Normalization

The internal Candidate model stores candidate_id, full_name, emails, phones, location, links, headline, years_experience, skills, experience, education, provenance, and overall_confidence. Normalizers lowercase and validate emails, convert phones to E.164-style strings, map countries to ISO alpha-2 codes, convert dates to YYYY-MM, and canonicalize skill aliases such as py -> Python, postgres -> PostgreSQL, and SpringBoot -> Spring Boot.

Area	Implemented Policy
Structured sources	ATS JSON blob and recruiter CSV export.
Unstructured sources	GitHub API-style snapshot and recruiter notes text parsed conservatively.
Conflict resolution	Source confidence ranking: ATS 0.90, CSV 0.85, GitHub 0.70, notes 0.60. Higher-confidence scalar wins; skills and links are merged without duplicates.
Provenance	Each extracted/normalized value records source, method, field, and confidence.
Configurable output	Projection config selects fields, renames paths with from, toggles provenance/confidence, normalizes output fields, and handles missing values with null, omit, or error.

Engineering Constraints

Deterministic: same inputs and config produce the same JSON. Robust: missing files, invalid JSON, malformed rows, bad emails, and bad phone numbers do not crash the run. Scalable: identity grouping uses indexed keys instead of pairwise comparisons. Explainable: README, tests, produced outputs, and this design document show the reasoning.

Edge Case	Handling
Same candidate in many sources	Merged into one record using email/phone identity keys; source evidence remains in provenance.
Bad email or phone	Normalizer returns null instead of guessing; wrong-but-confident data is avoided.
No usable identity	Skipped so the canonical output is not polluted with mystery records.
Different customer output shape	Use custom_config.json to emit renamed fields such as candidate_name, primary_email, and top_skills without code changes.

Deliberately Descoped

Live GitHub calls and resume PDF/DOCX parsing are not included to keep the demo deterministic and focused. They can be added later as new adapters because normalization, merge, projection, and validation are source-agnostic.

Core principle: wrong-but-confident is worse than honestly empty.